



July 24th, 2026, OpenQSE Workshop (QCUF 2026)



Two Abstraction Levels · Two Dialect Layers

A logical layer that reasons about error-corrected qubits, lowered to a physical layer of hardware qubits — each an MLIR dialect with its own types and passes.

Narasinga Rao Miniskar, Elaine Wong,
Seyong Lee, Leyton Ortega Vicente,
Jeffrey S Vetter, Travis Humble

miniskarnr@ornl.gov

Oak Ridge National Laboratory



U.S. DEPARTMENT
of **ENERGY**

ORNL IS MANAGED BY UT-BATTELLE LLC
FOR THE US DEPARTMENT OF ENERGY

DOE Acknowledgement

This presentation has been authored by UT-Battelle LLC under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<https://www.energy.gov/doe-public-access-plan>)

This work was supported by the U.S. Department of Energy, Office of Science under Contract No. DE-AC05-00OR22725, with funding from the Office of Advanced Scientific Computing Research's Accelerated Research in Quantum Computing Program's Quantum Science Center (QSC) project and resources of Oak Ridge National Laboratory's Experimental Computing Laboratory (ExCL).

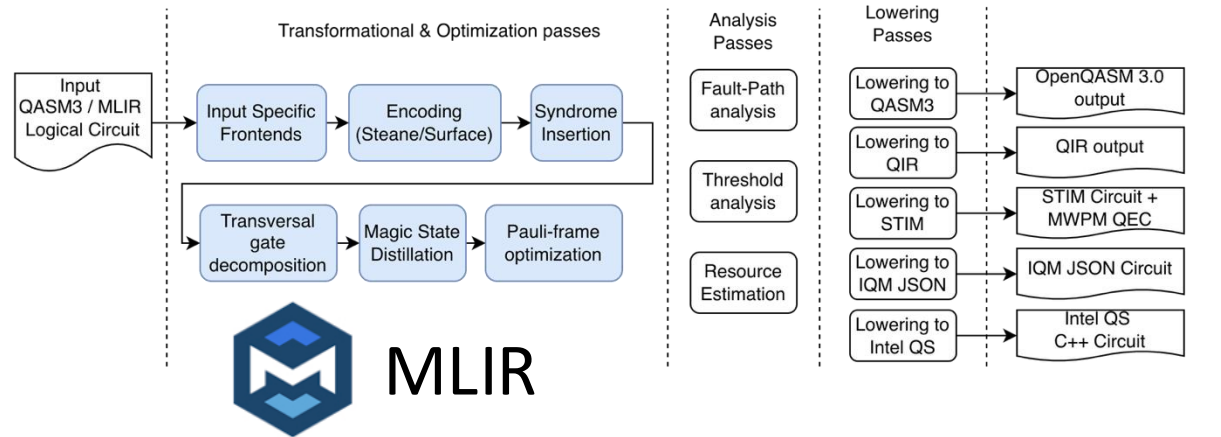
FTQC: An MLIR Dialect for Scalable Fault-Tolerant Quantum Computing

Narasinga Rao Miniskar, Elaine Wong, Seyong Lee, Vicente Layton Ortega, Yufei Ding, Jeff Vetter, Travis Humble

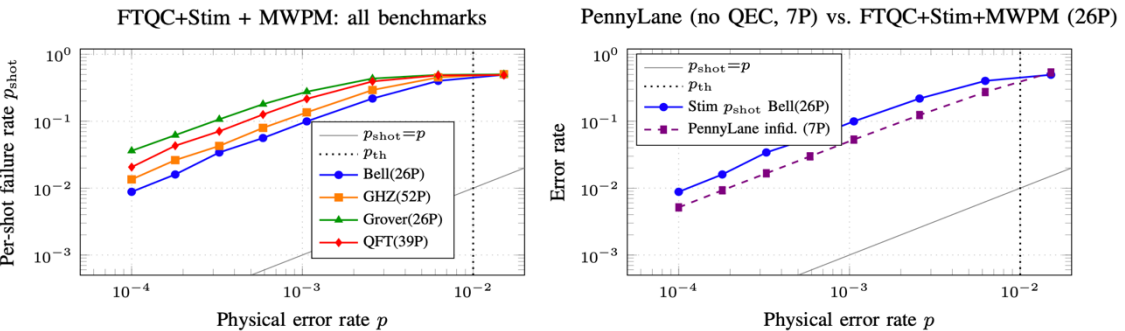
Objective

- Introduce FTQC, an open-source Multi-Level Intermediate Representation (MLIR) dialect and its framework.
- Provides a dedicated compilation pipeline for FTQC by treating error-correcting codes (ECC), syndrome extraction, and magic-state distillation as key intermediate representation (IR) constructs.

A Type-Safe MLIR Compiler Pipeline for Fault-Tolerant Quantum Computing



Logical Error Rates for FTQC+Stim+MWPM vs. PennyLane DM w/o QEC



Operation Categories and Counts

Category	Ops
Qubit lifecycle (init, encode, decode, dealloc)	5
Clifford gates (single- and two-qubit)	7
Non-Clifford gates (T, Toffoli, magic prep)	4
Syndrome extraction	5
Error correction & measurement	4
Pauli-frame management	3
Distillation & teleportation	3
Total	31

Example IR: Bell-state Preparation

```
1 func.func @bell_state_ft() -> (i1, i1) {
2   %lq0 = ftqc.init_zero : <<steane, 3>>
3   %lq1 = ftqc.init_zero : <<steane, 3>>
4   %h0 = ftqc.logical_h %lq0 : <<steane, 3>>
5   %c, %t = ftqc.logical_cnot %h0, %lq1
6   : (<<steane, 3>>, <<steane, 3>>)
7   -> (<<steane, 3>>, <<steane, 3>>)
8   %b0 = ftqc.logical_measure %c
9   : (<<steane, 3>>) -> i1
10  %b1 = ftqc.logical_measure %t
11  : (<<steane, 3>>) -> i1
12  func.return %b0, %b1 : i1, i1
13 }
```

- Provide frontend (QASM3, Qiskit, PennyLane QML) parsers to convert to MLIR and can be made inter-operable with other vendor IRs (QIR, Qiskit IRs, Quake, HUGR,..), and lower to vendor-specific IR/programming model requirements
- Enables to write vendor agnostic passes but can be integrated to call vendor specific passes as well
- WIP: Hardware aware FTQC and Integrate with Decoder

FTQC Compiler on top of MLIR, LLVM & the Python interface

PYTHON INTERFACE

`sim_*.py · ftqc.py · plot_*.py · *.venv`

Drives ftqc-opt, imports QASM, simulates (Aer / STIM / QIR) and submits to IQM hardware.

drives / consumes text IR

FTQC MLIR DIALECT + PASSES

`ftqc-opt · qasm3-import · ftqc.* / ftqc.qphys.*`

Types · Ops · Attrs (ODS / TableGen); encode → syndrome → transversal → lower to back ends.

built on

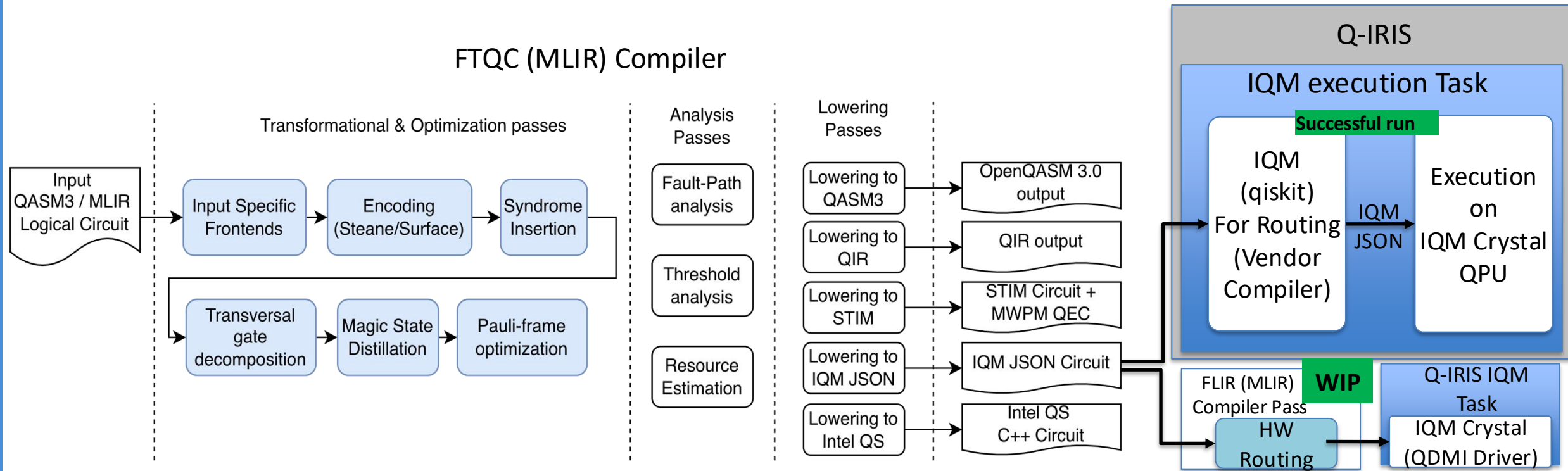
LLVM / MLIR 21+ INFRASTRUCTURE

`IR · PassManager · TableGen · FileCheck / lit`

Reusable compiler infrastructure the dialect is defined in and built on (C++17).

FTQC + Vendor compilation Passes and Execution on IQM

FTQC (MLIR) Compiler



WIP: Work in Progress

* Q-IRIS developed under MACH-Q project but extended it with IQM support and uses QDMI driver

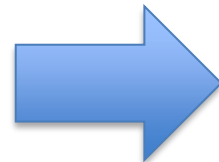
Two levels, bridged by one lowering pass

LEVEL 1 — LOGICAL

ftqc.*

`logical_qubit` · `syndrome` · `magic_state`

Error-corrected qubits, transversal gates, syndrome extraction, magic-state distillation, resource estimates.



`expand-physical`

LEVEL 2 — PHYSICAL

ftqc.qphys.*

`qphys_qubit`

Hardware qubits (data + ancilla) with physical gates & measurement — the substrate every back end emits.

Progressive lowering: source → Level 1 (encode, protect, compute, estimate) → `--ftqc-expand-physical` → Level 2 (physical circuit) → Hardware aware initial mapping → (Optional Level 3 dialects) IQM JSON / QASM3 / STIM → Vendor compilation passes

Reason about error-corrected qubits

Types: !ftqc.logical_qubit<code> !ftqc.syndrome<n> !ftqc.magic_state !ftqc.pauli_frame

Passes operating on these types:

- ftqc-steane-encode** encode each qubit into the Steane $[[7,1,3]]$ code; wrap gates in ec_round
- ftqc-insert-syndrome** insert steane_x/z_syndrome extraction between logical gates
- ftqc-transversal-gates** verify / expand logical gates into their transversal form
- ftqc-magic-distill** replace a noisy prepare_magic_t with 15-to-1 distillation (non-Clifford)
- ftqc-resource-estimate** annotate the function with logical/physical qubit & gate counts

Hardware qubits & physical gates

Type: !ftqc.qphys_qubit (a single hardware qubit; value-semantics)

Ops: qphys.alloc {id, role="data"|"x_anc"|"z_anc"} · h x z s t cx cz swap · measure

Passes operating on these types:

--ftqc-expand-physical bridge L1→L2: unfold each logical qubit into 7 data + 6 ancilla qphys qubits; lower gates & syndromes to physical circuits

--ftqc-qphys-to-iqm-json emit IQM JSON — locus QBn, native gates prx / cz / measure

--ftqc-qphys-to-qasm3 emit physical-qubit OpenQASM 3.0

(--ftqc-...-swap / routing) map onto a device coupling graph via qphys.swap [planned]

One circuit through every pass

Source (OpenQASM 3.0)

```
qubit[1] q; bit[1] c;  
► h q[0];  
c[0] = measure q[0];
```

auto-imported to Level 1 (ftqc.*)

```
%0 = ftqc.init_zero : <<steane, 3>>  
► %1 = ftqc.logical_h %0 : <<steane, 3>>  
%2 = ftqc.logical_measure %1 -> i1
```

Pipeline: steane-encode → resource-estimate (Level 1) → expand-physical → qphys-to-iqm-json (Level 2)

Wrap gates in error-correction rounds

```
%0 = ftqc.init_zero : <<steane, 3>>  
%1 = ftqc.logical_h %0 : <<steane, 3>>  
▶ %cq, %syn = ftqc.ec_round %1 rounds = 3 : <<steane, 3>> -> (<<steane, 3>>, <6>)  
%2 = ftqc.logical_measure %1 : (<<steane, 3>>) -> i1
```

Effect: each logical gate is followed by an `ftqc.ec_round` producing a 6-bit syndrome (`<6>` = 3 X + 3 Z stabilizers). Still fully logical — physical qubits are implicit.

Annotate the cost in-IR

```
func.func @hm() -> i1 {  
  ▶ ftqc.resource_annotation {logicalQubits = 1, physicalQubits = 13,  
  ▶ tGateCount = 0, cnotCount = 0, ecRounds = 0}  
  %0 = ftqc.init_zero : <<steane, 3>>  
  ...  
}
```

Effect: a Level-1 analysis pass records the estimate on the function — one Steane logical qubit needs 13 physical qubits (7 data + 6 ancilla). No IR gates change.

Data + ancilla qubits become MLIR

```

▶ %0 = ftqc.qphys.alloc {id = 0 : i64, role = "data"} : !ftqc.qphys_qubit
  ... (id 1-6 : 7 data qubits)
  %7 = ftqc.qphys.h %0 : !ftqc.qphys_qubit          // encoder + transversal H
  %o0, %o1 = ftqc.qphys.cx %7, %2 : !ftqc.qphys_qubit
▶ %17 = ftqc.qphys.alloc {id = 7 : i64, role = "x_anc"} : !ftqc.qphys_qubit
  %o0_16, %o1_17 = ftqc.qphys.cx %18, %10           // ancilla ← data
  %20 = ftqc.qphys.measure %19 {key = "sx"} : !ftqc.qphys_qubit -> i1
▶ %29 = ftqc.qphys.alloc {id = 10 : i64, role = "z_anc"} : !ftqc.qphys_qubit

```

Effect: the bridge pass unfolds the logical qubit into 7 data + 3 x_anc + 3 z_anc qphys qubits — explicit, labeled hardware qubits the back ends (and a routing pass) operate on.

Emit for the device

locus = QBn

QB1 - QB7 data qubits

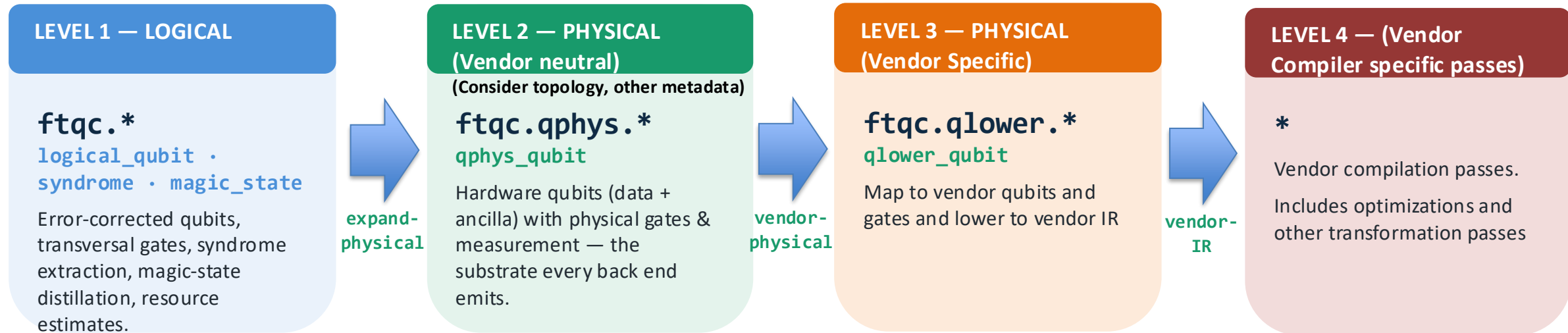
QB8 - QB13 X / Z ancillas

native prx · cz · measure

```
{ "name": "hm", "instructions": [
  { "prx", "locus": ["QB1"], "angle_t": 0.25, "phase_t": 0.25 } // H
  { "prx", "locus": ["QB1"], "angle_t": 0.5, "phase_t": 0 }
  { "cz", "locus": ["QB1", "QB3"] } // transversal encode
  ▶ { "prx", "locus": ["QB8"], ... } // X-ancilla
  ▶ { "cz", "locus": ["QB8", "QB1"] } // ancilla ← data
  ▶ { "measure", "locus": ["QB8"], "key": "m_QB8_.." } // syndrome
    { "measure", "locus": ["QB1"], "key": "m_QB1_.." } // readout
  ] }
```

Effect: physical qubits QB1–QB7 (data) and QB8–QB13 (ancilla) come straight from the qphys alloc ids — the emitter walks the physical IR, no layout re-derivation.

Proposing Multi levels



Progressive lowering: source → Level 1 (encode, protect, compute, estimate) →
 Level 2 (physical vendor neutral) → Hardware aware initial mapping →
 Level 3 (physical vendor specific) → Lower to hardware IR (Optional Level 3 dialects) IQM JSON / QASM3 / STIM → Vendor
 optimization and transformation passes in MLIR →
 Level 4 (Vendor Compiler specific passes)

Thank you